

# OpenSeimas V.4 Build Plan

## Entanglement

This document is the expanded entanglement reference for all OpenSeimas V.4 plan artifacts. It is intended to preserve the provenance of each draft, surface the shared strategy that binds them, and provide a high-detail crosswalk for product, engineering, and civic planning.

### Purpose of this document

The goal of this entanglement document is not just to list the files. It is to make explicit how the multiple draft plans relate to one another, where they agree, where they differ, and how the current repository can use all of them together. This version has been expanded to provide a detailed, character-rich guide that can support decision-making, implementation planning, and artifact retention.

### Current artifact catalog

The following files are the current V.4 plan artifacts discovered in docs/:

- docs/V4-build-plan-draft.md
- docs/V4-build-plan-final.md
- docs/V4-build-plan-final-overwrite-archive.md
- docs/V4-build-plan-manus-ai.md
- docs/V4-build-plan-grok.md
- docs/V4-build-plan-gemini-3.1-pro.md
- docs/V4-build-plan-gemini-3.5-flash-lite-prompt.md
- docs/V4-build-plan-chatgpt-5.6-sol.md
- docs/V4-build-plan-entanglement.md

The first eight are source artifacts and the last is this entanglement reference. The current file represents a richer, more detailed synthesis of those source artifacts.

# What this entanglement is meant to capture

This document captures the following dimensions:

1. **Model provenance** — which AI source produced each draft and how that source influenced tone and structure.
2. **Strategic alignment** — the shared product goals, architecture, and voter-centered priorities.
3. **Differentiation** — the unique emphasis of each artifact and the content value it brings.
4. **Practical use** — how to read and apply each draft in planning, ticketing, and stakeholder communication.
5. **Preservation guidance** — what to retain, what to treat as archival reference, and what to merge into a final implementation plan.

## Why this repo is suitable for V.4

OpenSeimas is suitable for the V.4 transition because the repository already contains the core structural elements needed for a voter-first civic observatory:

- `Seimas.v2/` as the consolidated Python backend and ingestion territory.
- `dashboard` as the existing React/Vite frontend prototype.
- `docs/archive/` for legacy prompt material and contextual history.
- `AGENTS.md` for agent role definitions and behavioral boundaries.
- `memory-bank.md` for voter-centered language and tone guidance.
- `cli.py` and `common.py` as existing operational pipeline anchors.

That existing structure means the project can evolve rather than rebuild. The V.4 focus is on enhancing and connecting what is already present.

## User empathy and why it matters

The typical Lithuanian voter we are designing for is a busy citizen who may be checking political news between work, school runs, or household tasks. They are not primarily a policy expert. They need a system that speaks clearly in Lithuanian, provides quick answers, and shows why a bill matters to their real life. This platform must make them feel seen, not preached to.

OpenSeimas V.4 should therefore convey respect for the user's time, explain practical impact, and avoid technical jargon. It should create the impression that

the person's vote is important, relevant, and connected to everyday priorities such as healthcare, education, transport, budget impact, and local services.

## Artifact summaries

### **docs/V4-build-plan-manus-ai.md**

**Source label:** Manus AI

**Tone:** strategic, product-focused, with strong emphasis on voter empathy and clarity.

**Core contribution:** A clear call to move OpenSeimas beyond research and into a live civic product. It emphasizes a clean backend pipeline, an agent layer, and a mobile-first interface. It also stresses non-partisan presentation and voter trust.

**Unique value:** This draft is especially useful for communicating the voter-facing mission and for framing the project as a product rather than a research experiment.

### **docs/V4-build-plan-grok.md**

**Source label:** Grok

**Tone:** polished and stakeholder-ready, with explicit notes on trust and evidence-backed guidance.

**Core contribution:** Reinforces the same product strategy while adding a sharper narrative about how the system should close the gap between Seimas operations and ordinary voters. It underlines trust and practical relevance.

**Unique value:** This draft is useful as a slightly more formal alternative to Manus AI, with strong language around evidence, recommendation quality, and general architecture.

### **docs/V4-build-plan-gemini-3.1-pro.md**

**Source label:** Gemini 3.1 Pro

**Tone:** formal, comprehensive, and suitable for stakeholders who need a concise summary of repo fit, implementation readiness, and execution risk.

**Core contribution:** Provides a strong “repository fit” argument and a product narrative that is easy to share with technical and civic reviewers. It highlights that the repo already has key assets like the pipeline and agent documentation.

**Unique value:** This file is the best candidate for formal stakeholder briefing and for grounding the project in the existing repo structure.

## **docs/V4-build-plan-gemini-3.5-flash-lite-prompt.md**

**Source label:** Gemini 3.5 Flash-Lite

**Tone:** meta and prompt-oriented rather than a finished plan.

**Core contribution:** This is the prompt used to generate detailed plans. It does not itself read like an execution plan, but it is valuable as a reference for future plan generation and model reuse.

**Unique value:** Keep this file as a prompt template. It documents the exact requirements and expectations used to generate professional migration plans. It is the best source for future plan regeneration or model-based iteration.

## **docs/V4-build-plan-chatgpt-5.6-sol.md**

**Source label:** Chat GPT 5.6 SOL

**Tone:** concise, practical, and focused on execution.

**Core contribution:** Offers the current most polished final-style plan, with direct mapping to repo locations and a strong phased roadmap. It is the version that appears to be the best candidate for final implementation guidance.

**Unique value:** This draft is the best starting point for converting plan narrative into an actual implementation backlog.

## **docs/V4-build-plan-draft.md**

**Source label:** initial canonical draft

**Tone:** structural and scaffold-like.

**Core contribution:** Provides the base format and initial idea structure that the other drafts build from.

**Unique value:** Use this file as the structural base for any final merge. It is the anchor for sections, and it should remain the canonical outline for what the final plan should cover.

**docs/V4-build-plan-final.md**

**Source label:** current final plan

**Tone:** narrative and consolidated.

**Core contribution:** Represents the synthesized final version of the plan, presumably intended as the primary artifact for stakeholder review.

**Unique value:** This file should be the main production plan, with the others used as supporting reference and extraction material.

**docs/V4-build-plan-final-overwrite-archive.md**

**Source label:** overwritten archive preservation

**Tone:** archival, historical.

**Core contribution:** Preserves the older version that was overwritten by accident. It is valuable for forensic understanding of how the plan evolved and for recovering any lost phrasing or requirements.

**Unique value:** Keep it as a historical artifact. Do not treat it as the active source for the final plan, but do reference it if there are questions about prior wording or scope decisions.

**Cross-artifact comparison matrix**

This matrix compares the files across key dimensions. It is a practical way to see the shared architecture and the variations in emphasis.

| Artifact | Product focus | Architecture detail | UX emphasis | Non-partisan focus | Repository fit | Use case              |
|----------|---------------|---------------------|-------------|--------------------|----------------|-----------------------|
| Manus AI | High          | Medium              | High        | High               | Medium         | Voter mission framing |
| Grok     | High          | Medium              | High        | High               | Medium         |                       |

| Artifact          | Product focus | Architecture detail | UX emphasis | Non-partisan focus | Repository fit | Use case                   |
|-------------------|---------------|---------------------|-------------|--------------------|----------------|----------------------------|
|                   |               |                     |             |                    |                | Stakeholder storytelling   |
| Gemini 3.1 Pro    | Medium        | High                | Medium      | High               | High           | Formal briefing            |
| Gemini 3.5 prompt | Low           | Low                 | Low         | High               | Low            | Prompt reuse               |
| ChatGPT 5.6 SOL   | High          | High                | High        | High               | High           | Final implementation guide |
| Draft             | High          | Medium              | Medium      | High               | High           | Structural anchor          |
| Final             | High          | High                | High        | High               | High           | Primary plan               |
| Final archive     | Medium        | Medium              | Medium      | High               | Medium         | Historical backup          |

## Detailed comparison of key dimensions

### Strategy and mission alignment

All drafts are aligned on the core mission: transform OpenSeimas into a voter-first civic observatory. This alignment is not shallow. It extends to shared statements such as:

- “Make voting choices understandable, relevant, and personally meaningful.”
- “Preserve non-partisan trust through transparent evidence.”
- “Deliver a mobile-first Lithuanian language UX.”
- “Use the pipeline as the source of truth.”

That repeated language means the project already has a strong strategic consensus. The remaining work is therefore not about choosing a different mission, but about operationalizing that mission.

## Architecture and execution

The drafts converge around a core system architecture:

- `Seimas.v2/pipeline/` for data ingestion and normalization.
- `Seimas.v2/main.py` for service and API delivery.
- `cli.py` as the operational runner for ingestion, validation, and maintenance.
- `dashboard` for the voter-facing interface.
- `AGENTS.md` and `memory-bank.md` for agent role definition and voter-centered tone.

This is the strongest shared consensus in the archive. The implication is clear: the repo already has an architecture; the work is to make it production-ready, explainable, and voter-centric.

## User experience and voter trust

Every plan emphasizes Lithuanian language UX as a trust and accessibility requirement. This is a critical convergence point. Using local language is not optional; it is a core design and credibility expectation. Likewise, every draft calls for low-friction onboarding, compact explanations, and source-backed output.

## Non-partisan and safety posture

All plans explicitly note that recommendations must remain non-partisan and explainable. This is a non-negotiable design constraint. The suggested implementation pattern is to build both a `SafetyAgent` and an `ExplainabilityAgent` so that the system can validate content and expose evidence.

## Roadmap and phasing

The phased roadmap is also a shared element. Differences exist in the exact durations and wording, but the high-level phases are consistent. The main path is:

1. archive/cleanup/stabilization
2. pipeline hardening
3. recommendation engine MVP
4. explainability/safety
5. frontend MVP

6. metrics and beta launch
7. post-beta iteration

This means the project already has a practical execution sequence. What remains is to convert this sequence into tickets, owners, acceptance criteria, and a calendar.

## How to use this document

This entanglement document is intentionally dense and reference-focused. Use it in the following ways:

1. **Plan consolidation:** Merge the shared strategy and phase sequence into a single implementation backlog.
2. **Artifact triage:** Keep the best elements of each draft while treating prompt artifacts and archives as reference only.
3. **Stakeholder briefing:** Use the “Gemini 3.1 Pro” and “ChatGPT 5.6 SOL” drafts for formal review, then cite the Manus AI and Grok drafts for product empathy.
4. **Future regeneration:** Use `V4-build-plan-gemini-3.5-flash-lite-prompt.md` as the future model prompt if you need to regenerate or expand the plan.
5. **Historical audit:** Keep `V4-build-plan-final-overwrite-archive.md` for forensic questions or lost wording.

## Recommended artifact usage pattern

- **Primary implementation plan:** `docs/V4-build-plan-final.md`
- **Supporting narrative:** `docs/V4-build-plan-grok.md`, `docs/V4-build-plan-manus-ai.md`
- **Formal briefing / stakeholder summary:** `docs/V4-build-plan-gemini-3.1-pro.md`
- **Plan regeneration prompt:** `docs/V4-build-plan-gemini-3.5-flash-lite-prompt.md`
- **Structural outline:** `docs/V4-build-plan-draft.md`
- **Backup archive:** `docs/V4-build-plan-final-overwrite-archive.md`



## Recommended consolidation approach

1. Extract structured objectives from all drafts. The shared objectives are the product backbone.
2. Merge the architecture layers and repository mapping into a single “Implementation Architecture” section.
3. Keep the phased roadmap in place, but add more detail for each phase in a separate project plan document.
4. Convert the detailed feature list into tickets for the frontend, backend, data, and safety streams.
5. Use the prompt artifact as a governance reference if the team decides to iterate with another model.

## Suggested next-stage artifacts

These are the most useful documents to produce after this entanglement reference:

- docs/V4-build-plan-implementation.md: a merged, ticket-ready plan with exact deliverables.
- docs/V4-build-plan-risk-register.md: a risk matrix drawn from the shared risk items.
- docs/V4-build-plan-metrics-schema.md: a metrics definition sheet aligned with the success measures.
- docs/V4-build-plan-content-guidelines.md: a writing style guide based on memory-bank.md and the voter empathy language.

## Detailed feature extraction from the drafts

The drafts collectively recommend the following product capabilities:

- **Recommendation cards** with a headline, three bullet reasons, a confidence layer, and source-provenance markers.
- **Compact Lithuanian explanations** that prioritize plain language, a conversational tone, and short sentences.
- **Explicit confidence and uncertainty visualization** to help users understand when guidance is strong versus tentative.
- **Source links, provenance badges, and evidence citations** so users can verify claims.

- **Lightweight preference onboarding** with optional priorities that do not require long profiles or invasive tracking.
- **Feedback capture and trust surveys** to measure how convincing the guidance is.
- **Accessibility-first mobile layout** so the interface is usable on phones, in low-light, and by older voters.
- **Non-partisan reading mode** that lets users view the same data without recommendation framing.
- **“Why this matters to you”** messaging that connects bills to daily life rather than abstract policy.
- **Freshness indicators** so users know whether the data is current.
- **Historical context and trend summaries** that explain how a current decision fits into past parliamentary behavior.

Each of these features appears in at least two of the source drafts. That cross-draft repetition makes them strong candidates for the MVP and for prioritized implementation.

## Implementation notes drawn from all drafts

### Python backend

- Use `Seimas.v2/pipeline/common.py` for shared import, config, and logging utilities.
- Keep `Seimas.v2/pipeline/cli.py` as the operational command line entrypoint. It should support commands such as `ingest`, `validate`, `export`, and `clean`.
- Add a typed `Seimas.v2/agents/` package with modules for each agent role.
- Use Pydantic models for normalized artifacts such as `Bill`, `Vote`, `Member`, `Party`, and `SourceCitation`.

### API contract

Design the API with explicit, stable endpoints:

- `GET /recommendations`: returns a list of voter guidance items, including headline, reasons, confidence, sources, and `topicTags`.
- `GET /bills/{billId}`: returns bill metadata, current stage, sponsors, associated votes, and provenance.

- GET /preferences: returns available personalization options.
- POST /feedback: captures user trust rating, comprehension rating, and free-form comments.

## Frontend architecture

- Base the dashboard on React + Vite.
- Use modular components for RecommendationCard, BillDetail, OnboardingStep, SourcePanel, and FeedbackForm.
- Keep the first mobile screen lightweight: headline, reason bullets, and a single “read more” action.
- Make the onboarding flow optional and fast, with persistent local storage for preferences.

## Data workflow

- Ingest raw parliamentary feeds into normalized JSON artifacts.
- Store clean data with versioning or timestamped snapshots.
- Use cli.py to run daily or event-driven ingests.
- Surface ingest freshness and failure state in the API.
- Use the pipeline to calculate relevance scores and to feed the agent layer.

## Metrics and measurement

The plans recommend six measurable metrics. These should be tracked in a lightweight dashboard or analytics table.

- **Comprehension:** percentage of users who can correctly paraphrase a recommendation. Target:  $\geq 75\%$  in early tests.
- **Trust:** average user trust score on a 5-point scale. Target:  $\geq 4.0$ .
- **Action:** percentage of users who open source details, save a bill, or mark it useful. Target: sufficient to show meaningful exploration.
- **Feedback:** completion rate of trust and usefulness prompts. Target: 20–30% of active sessions.
- **Data freshness:** percentage of recommendations based on data updated within the expected ingest window. Target:  $\geq 95\%$ .
- **Provenance coverage:** percentage of recommendations with full source citations and explainability notes. Target: 100%.

## Example metrics matrix

| Metric              | Why it matters                                             | Target                   |
|---------------------|------------------------------------------------------------|--------------------------|
| Comprehension       | Ensures users actually understand guidance                 | ≥ 75%                    |
| Trust               | Measures credibility and acceptance                        | ≥ 4.0/5                  |
| Action              | Indicates whether guidance drives exploration or follow-up | Meaningful click rate    |
| Feedback            | Provides signal for iteration                              | 20–30% completion        |
| Data freshness      | Prevents stale recommendations                             | ≥ 95% of records updated |
| Provenance coverage | Keeps guidance auditable                                   | 100% citations           |

## Risk register

These risk items are drawn from the source artifacts and from the repository context.

### Perceived bias

- **Why it matters:** Civic trust collapses if users feel the platform is pushing a partisan agenda.
- **Mitigation:** build explicit non-partisan guardrails, use a safety review step, and expose provenance for every claim.

### Uncertain recommendations

- **Why it matters:** Users may ignore the system if they feel it is guessing.
- **Mitigation:** display uncertainty clearly and provide “insufficient evidence” outcomes rather than forced conclusions.

### Stale or incomplete data

- **Why it matters:** Outdated or missing information can mislead voters.

- **Mitigation:** track ingest freshness, fail fast on incomplete sources, and communicate data age to users.

## Low user trust or adoption

- **Why it matters:** The product can exist without impact if few voters use it.
- **Mitigation:** test language and experiences with representative Lithuanian users early, and iterate on clarity.

## Engineering scope creep

- **Why it matters:** The project can lose focus and delay launch.
- **Mitigation:** keep the MVP narrow, prioritize roadmap phases, and gate new features with explicit user value.

# Governance recommendations

Immediate decisions:

- Approve the agent-based architecture and the specific boundaries for VoterGuideAgent, PersonalizationAgent, ExplainabilityAgent, SafetyAgent, and PipelineAgent.
- Confirm the MVP scope: recommendation cards, onboarding, explainability, and feedback.
- Assign owners for backend, frontend, product, and civic review.
- Establish review checkpoints for architecture, beta readiness, and trust validation.

Concrete next deliverables:

1. `Seimas.v2/agents/voter_guide.py` prototype.
2. `docs/V4-build-plan-draft.md` refinement into an implementation backlog.
3. CI ingest workflow and release checklist.

## Appendix: artifact metadata

The file artifacts were created from the session transcript and are preserved in the repository as both working and archival documents. They should remain available for audit and for future plan regeneration.

### Recommended preservation status

- **Active reference:** docs/V4-build-plan-final.md, docs/V4-build-plan-gemini-3.1-pro.md, docs/V4-build-plan-chatgpt-5.6-sol.md
- **Supporting reference:** docs/V4-build-plan-manus-ai.md, docs/V4-build-plan-grok.md, docs/V4-build-plan-draft.md
- **Archive reference:** docs/V4-build-plan-final-overwrite-archive.md, docs/V4-build-plan-gemini-3.5-flash-lite-prompt.md

## Final guidance

This expanded entanglement document is intended to support a practical transition from multiple AI-generated drafts into a single coherent execution plan. Use the shared strategy and architecture consensus as the foundation, and treat the unique drafts as complementary sources of nuance, product empathy, and stakeholder language.

The most important directional rule is simple: keep the project voter-first, Lithuanian-language, non-partisan, and explainable.